2024 IEEE International Conference on Multimedia and Expo (ICME) | 979-8-3503-9015-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICME57554.2024.10687950

TRAINING-FREE ROBUST NEURAL NETWORK SEARCH VIA PRUNING

*Qiancheng Yang[1], Yong Luo[1], Bo Du[2]**

[1]National Engineering Research Center for Multimedia Software, School of Computer Science, Wuhan University, Wuhan, China
[2]Hubei Luojia Laboratory, Wuhan, China

ABSTRACT

Adversarial examples widely exist in visual tasks, which look almost the same as normal images but can induce neural networks to make completely wrong predictions. This exposes significant risks to deploying deep learning to visual tasks. The robustness of neural networks to adversarial images is shaped by multiple factors entangled together, including adversarial training strategies, model architectures, hyper-parameters, and optimization algorithms. This makes searching for robust architectures a significantly complex task. In this paper, we propose a Robust Training-free Pruning neural architecture search (RTP-NAS) framework that endeavors to disentangle the influence of architectures from other factors. We adopt Universal Adversarial Perturbations (UAPs) to construct a transferable adversarial data space across different network architectures. RTP-NAS further employs two adversarial training-free indicators, the condition number of the neural tangent kernel (NTK) and the number of linear regions calculated on the adversarial data space, to measure the intrinsic adversarial robustness of candidate architectures. Comprehensive experiments show that the proposed method achieves superior performance to other existing approaches, with a significantly lower computational cost. The code will be released publicly.

***Index Terms*—** Adversarial robustness, NAS, NTK, linear regions

1. INTRODUCTION

Deep learning has been successfully applied to a wide variety of visual applications, including image retrieval, autonomous driving, face recognition, and object detection. Despite these successes, deep neural networks are vulnerable to adversarial attacks. By adding carefully crafted imperceptible noise to normal data, an attacker can easily create adversarial images that fool deep neural networks into unexpected behaviors [1]. To tackle the challenge of adversarial attacks, existing works have designed various training strategies, such as adversarial training, to enhance the robustness of deep neural networks towards adversarial images [2].

Beyond the realm of training algorithms, some recent works have proposed searching for neural network architectures that are inherently robust to adversarial examples by leveraging neural architecture search (NAS) algorithms [3, 4]. Under the NAS paradigm, one would repeatedly search for candidate architectures and evaluate their adversarial robustness to find the most robust one. However, adversarial robustness evaluation, typically achieved through adversarial training, involves multiple intertwined factors. These include adversarial training strategies, model architecture hyper-parameters, and optimization algorithms. Such an adversarial training-based evaluation approach may conceal the inherent robustness of neural architectures and fail to search for robust architectures using NAS. This raises the following question: *Can we disentangle the influence of network architecture from other factors in the pursuit of robust neural networks?*

In this paper, to answer this question, we propose a Robust Training-free Pruning NAS (RTP-NAS) framework, which employs a training-free NAS strategy [5] to search for architectures of strong intrinsic robustness without the interference of irrelevant factors. Specifically, in the original training-free NAS paradigm, two indicators, the condition number of the neural tangent kernel (NTK) [6] and the number of linear regions, are leveraged to measure the trainability and expressivity of candidate architectures without actually training them. This, therefore, enables architecture search to be highly efficient.

This motivates us to calculate two indicators on the input space of adversarial examples to disentangle and directly quantify the intrinsic adversarial robustness of the searched architectures. Compared with existing robust architecture searching approaches, our RTP-NAS evaluates the intrinsic robustness of neural architectures more efficiently. It avoids the time-consuming and potentially oscillating adversarial training, simultaneously reducing search time and enhancing the robustness of the searched architecture.

A critical aspect of RPT-NAS involves constructing an adversarial sample space that is transferable across different structures, which is utilized to compute metrics for assessing robustness. In this work, we employ universal adversarial perturbations (UAPs) to transform normal images into adversar-

*Corresponding author

979-8-3503-9015-5/24/$31.00 ©2024 IEEE
Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on March 25,2026 at 05:33:42 UTC from IEEE Xplore. Restrictions apply.

ial ones, approximating the corresponding input space [7, 8]. Compared with constructing adversarial input space using the projected gradient descent (PGD) attack strategy, adopting UAPs in our RTP-NAS offers two advantages. First, UAPs can be pre-generated using pre-trained models without saving adversarial images, reducing the cost of calculating adversarial training-free indicators. Second, adversarial images created with UAPs exhibit remarkable transferability across various neural architectures, enhancing the capability of training-free indicators to measure intrinsic architecture robustness.

Our contribution can be summarized as follows:

- We propose RTP-NAS, a novel NAS framework to search for discovering robust neural architectures. We design two unique training-free indicators, calculated on the adversarial input space constructed using UAPs. This method effectively disentangles and directly measures the intrinsic robustness of architectures, ensuring the identification of highly robust structures without the influence of extraneous factors.
- We explore how to construct an effective adversarial example space. By utilizing UAPs to construct an adversarial input space, we ensure consistency in robustness evaluation across different structures.
- We provide valuable insights by conducting an in-depth analysis of the architectures identified by our search. Based on this analysis, we propose a simple yet effective strategy to further enhance the performance of these architectures.

Comprehensive experiments were conducted on two benchmark datasets, CIFAR-10 and CIFAR-100 [9], to empirically validate the effectiveness of our RTP-NAS framework in identifying robust neural architectures. The results of these experiments demonstrate the superiority of the architectures found through RTP-NAS when compared with their competitive counterparts.

2. PRELIMINARIES

In this section, we will introduce the concept of NTK and linear regions.

2.1. Condition number of neural tangent kernel

A critical point in measuring the quality of a neural network is its trainability. However, characterizing the gradient-based training dynamics of the model is challenging owing to the typically high-dimensional non-convex loss surfaces that govern the optimization [10, 11]. Such a characterization is provided in [12] by analyzing the spectrum of the neural tangent kernel (NTK). The finite width NTK is defined by:

$$\mathcal{H}\left(x, x'\right) = J\left(x\right) J\left(x'\right)^{T}, \tag{1}$$

where $J_{i\alpha}(x) = \partial_{\mathcal{H}\alpha} z i^{L}(x)$ represents the gradient of the i-th output of the last layer with respect to parameter α, evaluated at a point x. The main result in [6] shows that the NTK converges to a deterministic kernel and remains constant during training. The authors then demonstrate that the expected outputs of an infinitely wide network can be derived by solving the ordinary differential equation (ODE):

$$\mu_t\left(X_{\text{train}}\right) = \left(\mathbf{Id} - e^{-\eta \mathcal{H}_t}\right) Y_{\text{train}}. \tag{2}$$

Here, $\mathbf{Id}$ is an identity matrix, and $\mathcal{H}_t$ denotes the NTK at the step t. As t approaches infinity, Eq. (2) reduces to $\mu_t\left(X_{\text{train}}\right) = Y_{\text{train}}$ and can be further written as:

$$\bar{\mu}_t\left(X_{\text{train}}\right)_i = \left(\mathbf{Id} - e^{-\eta \lambda_{t,i}}\right) \bar{Y}_{\text{train},i}, \tag{3}$$

where $\lambda_{t,0} \geq \cdots \geq \lambda_{t,m}$ are the eigenvalues of $\mathcal{H}_t$, and $\bar{\mu}_t\left(X_{\text{train}}\right)$, $\bar{Y}_{\text{train}}$ are the mean prediction and labels respectively. It has been hypothesized in [10, 13] that the maximum feasible learning rate scales as $\eta \sim 2/\lambda_0$, which is empirically verified by [12]. Plugging this scaling for η into Eq. (2), we can find that the smallest eigenvalue will converge exponentially at a rate given by the condition number $\kappa = \lambda_0/\lambda_m$. A diverging condition number indicates that the neural network becomes untrainable, and this can be adopted as a metric for measuring the network's trainability. Finding the network with a smaller κ is essential for effectively training the neural networks (NNs).

2.2. Linear regions

Another factor that affects the performance of a neural network is its expressivity. It is known that the composition of piece-wise linear functions is still piece-wise linear. Every feed-forward ReLU NN with certain parameters can be seen as a piece-wise linear function [14]. Hence, the expressivity of a ReLU NN can be quantified by the maximal number of subspaces into which we can separate input space. Following [15], we have the following definitions of Activation Patterns and Linear Regions:

Definition 1 (Activation Patters and Linear Regions)**.** *Let $\mathcal{N}$ be a ReLU CNN with L hidden convolutional layers. An activation patter of $\mathcal{N}$ is a function $\mathcal{P}$ from the set of neurons to $\{1, -1\}$, i.e., for each neuron z in $\mathcal{N}$, we have $\mathcal{P}(z) \in \{1, -1\}$. Let θ be a fixed set of parameters (weight and biases) in $\mathcal{N}$, and $\mathcal{P}$ be an activation pattern. The region corresponding to $\mathcal{P}$ and θ is:*

$$\begin{aligned}\mathcal{R}(\mathcal{P};\theta) :=\{&X^0 \in \mathbb{R}^{C\times H\times W} : \\ &z\left(X^0;\theta\right)\cdot \mathcal{P}(z) > 0, \quad \forall z \in \mathcal{N}\},\end{aligned} \tag{4}$$

where $z\left(X^0;\theta\right)$ is the pre-activation of a neuron z. A linear region of $\mathcal{N}$ at θ is a non-empty set $\mathcal{R}(\mathcal{P};\theta) \neq \emptyset$ for the same activation pattern $\mathcal{P}$. Let $\mathcal{R}_{\mathcal{N},\theta}$ denotes the number of linear regions of $\mathcal{N}$ at θ, i.e., $R_{\mathcal{N},\theta} := \#\{\boldsymbol{R}(\boldsymbol{P};\theta) : \boldsymbol{R}(\boldsymbol{P};\theta) \neq \emptyset$ for some activation pattern $\boldsymbol{P}\}$.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on March 25,2026 at 05:33:42 UTC from IEEE Xplore. Restrictions apply.

It is found in [16] that the network with more linear regions performs better because of its stronger expressivity.

3. APPROACH

In this section, we introduce the core components of our proposed RTP-NAS method. We first generate universal adversarial perturbation (UAP) [8] using the model pre-trained on the training set. Then we construct transferable adversarial input space by adding UAP to the original inputs. Afterward, we prune the supernet using the adversarial condition member k_A and linear regions calculated in the adversarial input space. The details are depicted as follows.

3.1. Crafting universal adversarial perturbations

Given a dataset $\mathcal{D}_S = \{(x_i, y_i) \mid i = 1, 2, \cdots, N\}$, which contains N standard input-label pairs. Adversarial examples $x_i' \in \mathcal{D}_A$ can be crafted by adding perturbations δ to the original inputs, causing the target model $\mathcal{C}$ to make erroneous predictions.:

$$\mathcal{C}\left(x_i + \delta\right) \neq y_i, \qquad \text{s.t.} \quad \|\delta\|_p \leq \epsilon, \tag{5}$$

where perturbation δ is bounded by an l_p-norm and constraint ϵ is imperceptible to human eyes. δ can be crafted by various attack methods, such as the Fast gradient Sign Method (FGSM) [17], Projected Gradient Descent (PGD). In Appendix Section 1, we provide an introduction to some of the methods. Distinct from typical perturbations, universal adversarial perturbations are sample-agnostic. They can be defined as follows:

$$\begin{aligned} \mathcal{C}\left(x_i + v\right) \neq y_i \qquad & \text{s.t.} \quad \|\delta\|_p \leq \epsilon, \\ & \text{for most } x_i \in D_S\ . \end{aligned} \tag{6}$$

For dataset D_S, we can easily obtain a standard trained model $\mathcal{C}$ on it. To generate UAPs, the PGD algorithm is adopted to generate perturbations consisting of untargeted features similar to [7]. We define loss function $\mathcal{L}_U$ for the generation of untargeted UAPs as:

$$\mathcal{L}_U = \max\{C_{gt}\left(x_i + v\right) - \max_{i \neq gt} C_i\left(x_i + v\right), -\beta\}, \tag{7}$$

where β is the confidence value, $C_{gt}\left(x_i + v\right)$ refers to the logit value of the ground truth label and $\max_{i \neq gt} C_i\left(x_i + v\right)$ refers to the maximum logit value other than the ground truth label. In the process of UAP generation, $C_{gt}\left(x_i + v\right)$ reduces continuously. Our goal is to reduce the confidence predicted to the correct class on the perturbed input $x_i + v$ and increase the confidence predicted to the wrong category. A comprehensive flowchart detailing the algorithmic process is included in the appendix for further reference. The resulting adversarial input space can be denoted as:

$$\begin{aligned} \mathcal{D}_A = & \{x_i + v, y_i \mid i = 1, 2, \cdots, N\}, \\ & \text{for each } x_i \in \mathcal{D}_{\mathcal{S}}. \end{aligned} \tag{8}$$

We generate UAP from the pre-trained model given the perturbation constraint ϵ. For different pre-trained models and constraints, we can obtain different UAPs for constructing the adversarial input space. An example is illustrated in the appendix.

3.2. Pruning process

After constructing the adversarial input space, we search for the network that has the strongest expressivity and trainability from the constructed searching space. Fig. 1 illustrates the main procedure, where the construction of searching space and pruning process is introduced as follows.

By transforming the standard input space to adversarial input space constructed in Section 3.1, we propose adversarial NTK to evaluate how effective a network can be optimized via gradient descent when conducting adversarial training:

$$\mathcal{H}_A\left(x + v, x' + v\right) = J(x + v) J\left(x' + v\right)^T. \tag{9}$$

Now, we can obtain the adversarial condition number κ of $\mathcal{H}_A$. Combining it with the linear regions as mentioned above, we obtain the metrics for pruning. The principle is that we tend to find the network that has small κ_A and more linear regions $R_{\mathcal{N},\theta}$. In other words, we prefer a network that is expressive and easy to conduct adversarial training.

However, for the DARTS searching space, supposing there are E edges in each cell and $\mathcal{O}$ possible operators for each edge, we will have about O^E possible candidate architectures. Some existing works randomly sample the architecture from the searching space and evaluate its performance [4], which is costly. Following [22, 5], we search for the target network by pruning the operators according to the importance of each edge. Since we prefer the network that has small κ_A and large $R_{\mathcal{N},\theta}$. The edge that, after being pruned, reduces the number of linear regions as few as possible and the value of κ_A as much as possible, should be less important.

As shown in Fig. 1, we initialize the network $\mathcal{N}_0$ using the Kaiming Norm Initialization [23] strategy. Then for each remained operator $o_i \in \mathcal{O}$ in edge E_j, we calculate the change in condition number $\Delta\kappa_A$ and $\Delta R_{\mathcal{N},\theta}$ after removing it:

$$\Delta\kappa_{A,t,o_i} = \kappa_{\mathcal{N}_t} - \kappa_{\mathcal{N}_t \backslash o_i}, \tag{10}$$

$$\Delta R_{t,o_i} = R_{\mathcal{N}_t} - R_{\mathcal{N}_t \backslash o_i}, \tag{11}$$

We sort $\Delta\kappa_{A,t,o_i}$ in descending order and $\Delta R_{t,o_i}$ in ascending order respectively:

$$R_d(\Delta\kappa_{A,t,o_i}) = \text{rank of the descending order} \tag{12}$$

$$R_a(\Delta R_{t,o_i}) = \text{rank of the ascending order} \tag{13}$$

The higher $\Delta\kappa_{A,t,o_i}$, the more likely we will prune o_i; The lower $\Delta R_{t,o_i}$, the more likely we will prune o_i. Then we score each operator according to its rank:

$$Score\left(o_i\right) = R_d(\Delta\kappa_{A,t,o_i}) + R_a(\Delta R_{t,o_i}). \tag{14}$$

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on March 25,2026 at 05:33:42 UTC from IEEE Xplore. Restrictions apply.

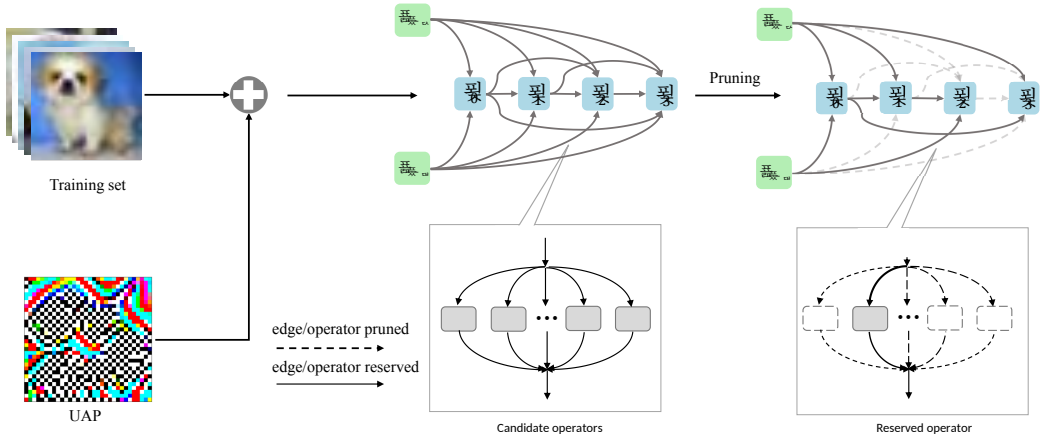
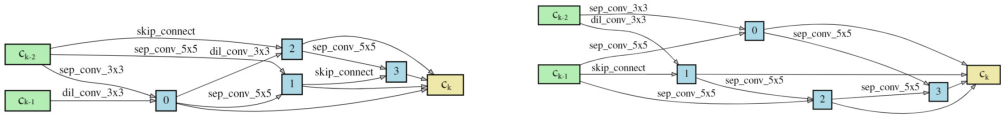


Fig. 1. Main procedures of the proposed RTP-NAS. We craft universal adversarial perturbations from the pre-trained model using the training set. Then we construct adversarial input space by adding UAP to the origin input. Afterward, the condition number of adversarial NTK and the number of linear regions is computed for scoring edges. In each round, we prune the operators according to the minimal score.



(a) Normal Cell (b) Reduction Cell

Fig. 2. Visualization of cells searched by RTP-NAS. The network is stacked by the normal cells and the reduction cells.

For each $e_j \in E$ in $\mathcal{N}$, we find the operator o_m that has the minimal score and prune it:

$$\mathcal{N}_{t+1} = \mathcal{N}_t \backslash o_m, \tag{15}$$

we will stop pruning until $\mathcal{N}_t$ becomes the single-path, i.e., each edge keeps two operators. An example of the final searched architecture is shown in Fig. 2.

4. EXPERIMENTS

In the following, we conduct comprehensive experiments on CIFAR-10 and CIFAR-100 to demonstrate the effectiveness of the proposed RTP-NAS. Implementation details and analysis of the architectures are listed in the appendix.

4.1. Against white-box attacks

Main results. We compare RTP-NAS with classical networks, general NAS algorithms including DARTS [18], PC-DARTS [19], and robust NAS algorithms including RACL [3], AdvRush [20] and RNAS [21]. All architectures are adversarially trained and evaluated under white-box attacks on CIFAR-10. As shown in the left part Tab. 1, RTP-NAS achieves the highest standard accuracy (without attacking) and robust accuracy under different attacks compared to the other models. Besides, RTP-NAS requires neither training the supernet nor training candidate architectures during the search. Hence, the search cost can be significantly reduced, and we can complete the search in only one hour.

Transferability to other datasets. We transfer the architecture searched on CIFAR10 to CIFAR-100. Results in Tab. 1 show that the searched architecture generalizes well across different datasets. On CIFAR100, RTP-NAS achieves a clean accuracy of 60.71%, which is only a little lower than DenseNet121 but with much fewer parameters, and it achieves the highest robust accuracy under different attacks compared to the other models.

4.2. Ablation study

In this section, we conduct ablation studies on the utilized search metrics and UAPs. Experimental results are reported in Tab. 2 and Tab. 3.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on March 25,2026 at 05:33:42 UTC from IEEE Xplore. Restrictions apply.

Table 1. We evaluate the adversarial robustness of all architectures adversarially trained on CIFAR-10 and CIFAR-100. The best result in each column is in bold. PGD^{20} and PGD^{100} refer to PGD attacks with 20 and 100 steps respectively, and AA refers to standard AutoAttack.

Model	Params	CIFAR10					CIFAR100				
		Clean	FGSM	PGD^{20}	PGD^{100}	AA	Clean	FGSM	PGD^{20}	PGD^{100}	AA
ResNet-18	11.2M	80.58%	53.60%	46.49%	45.95%	42.70%	41.25%	21.78%	19.73%	19.63%	15.60%
VGG16	14.7M	80.78%	54.17%	47.57%	47.15%	43.24%	51.94%	27.69%	23.85%	23.67%	21.17%
DenseNet-121	7.0M	84.88%	59.69%	51.76%	51.33%	49.96%	**61.44%**	32.86%	28.51%	27.99%	25.38%
DARTS [18]	3.3M	84.16%	58.14%	51.76%	51.33%	49.69%	56.69%	32.73%	28.81%	28.50%	22.37%
PC-DARTS [19]	3.4M	84.59%	58.55%	51.14%	50.92%	48.52%	59.18%	32.61%	28.06%	27.89%	22.06%
AdvRush [20]	4.2M	84.26%	58.76%	52.33%	51.46%	49.05%	59.68%	33.13%	28.98%	28.76%	26.46%
RACL [3]	3.6M	83.69%	58.39%	51.41%	51.11%	48.79%	59.65%	32.75%	28.90%	28.69%	25.67%
RNAS [21]	3.6M	83.94%	56.09%	50.77%	50.49%	48.86%	54.32%	29.05%	28.73%	29.50%	25.78%
RTP-NAS	3.7M	**85.70%**	**59.92%**	**53.11%**	**52.76%**	**50.14%**	60.71%	**34.92%**	**31.04%**	**30.92%**	**27.15%**

Influence of search metrics. We performed pruning using different metrics and evaluated the performance of search architectures to validate their effectiveness. The ablation analysis is shown in Tab. 2. Compared to the search according to only κ_A or $R_{\mathcal{N},\theta}$, RTP-NAS performs better in terms of both standard and adversarial evaluation. This indicates that not only expressive power of the model matter, but the trainability also plays an important role in model performance. Comparing the results between the first row and the second row, the adversarial and standard accuracies are improved by replacing κ with κ_A. This can be further validated according to the results in the bottom two lines.

Influence of different UAPs. We generate universal adversarial perturbations from three different pre-trained models to construct adversarial input space. Results are reported in Tab. 3. The ϵ refers to the constraint of UAPs. The fooling ratio indicates the percentage of images on which the pre-trained model changes its prediction label after the images are perturbed. Clean and robust accuracies are the evaluation of architectures searched within the adversarial input space constructed by the corresponding source architectures.

Comparing the effect of UAPs generated by different architectures on the searched result, we can find that the adversarial input space constructed by the VGG family performs better than ResNet. Although UAPs generated from ResNet56 achieve the highest fooling ratio, the UAPs generated by VGG16 have the strongest transferability, which is demonstrated in the experiments of [8] (Tab. 5).

By constructing transferable adversarial input space, we can evaluate the robustness of candidate models with different architectures. Results of Tab. 3 also show that the adversarial robustness of searched models gets improved when the constraint is increased. Given larger constraint ϵ, UAPs can transfer better, which can further guarantee the abilities of adversarial training-free indicators for measuring the intrinsic robustness of architectures.

Table 2. Influence of different metrics for searching

Metric	Clean	PGD20
κ	79.68%	48.11%
κ_A	81.41%	49.72%
$R_{\mathcal{N},\theta}$	78.88%	49.40%
$\kappa + R_{\mathcal{N},\theta}$	79.99%	50.21%
RTP-NAS	85.70%	53.11%

Table 3. We construct different adversarial input spaces by adopting different pre-trained models under the constraints of $10/255$ and $20/255$. Then we search for the architectures from each space and evaluate their performance to analyze the influence of the UAPs.

Source arch	ϵ	Fooling ratio	Clean	PGD20
VGG16	10/255	89.66%	84.80%	51.05%
VGG19		82.94%	79.01%	48.40%
ResNet56		94.07%	81.11%	49.82%
VGG16	20/255	89.66%	85.70%	53.11%
VGG19		90.03%	84.13%	52.07%
ResNet56		94.07%	82.98%	50.62%

5. CONCLUSIONS

In this paper, RTP-NAS is proposed to search for robust neural architectures based on training-free pruning. The key to our RTP-NAS is to decouple measuring the intrinsic robustness of a network from adversarial training. To achieve this goal, we adopt universal adversarial perturbations that have

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on March 25,2026 at 05:33:42 UTC from IEEE Xplore. Restrictions apply.

strong transferability among different network architectures to construct adversarial input space. Then we adopt adversarial NTK and the number of linear regions to measure how well the model performs in this space. Comprehensive experiments validate the effectiveness of our RTP-NAS, providing an efficient method for searching for robust architectures. Our analysis of the model structure also provides some insights and suggestions for improving the robustness of the model.

Acknowledgment

This work is supported in part by the National Key Research and Development Program of China (2023YFC2705700), the National Natural Science Foundation of China (Grant No. U23A20318 and 62276195), and the Fundamental Research Funds for the Central Universities (No. 2042024kf0039).

6. REFERENCES

[1] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian J. Goodfellow, and Rob Fergus, "Intriguing properties of neural networks," in *ICLR 2014,*, Yoshua Bengio and Yann LeCun, Eds., 2014.

[2] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu, "Towards deep learning models resistant to adversarial attacks," in *ICLR 2018*.

[3] Minjing Dong, Yanxi Li, Yunhe Wang, and Chang Xu, "Adversarially robust neural architectures," *CoRR*, vol. abs/2009.00902, 2020.

[4] Chaitanya Devaguptapu, Devansh Agarwal, Gaurav Mittal, Pulkit Gopalani, and Vineeth N Balasubramanian, "On adversarial robustness: A neural architecture search perspective," in *ICCV*, 2021, pp. 152–161.

[5] Wuyang Chen, Xinyu Gong, and Zhangyang Wang, "Neural architecture search on imagenet in four GPU hours: A theoretically inspired perspective," in *ICLR 2021*, 2021.

[6] Arthur Jacot, Franck Gabriel, and Clément Hongler, "Neural tangent kernel: Convergence and generalization in neural networks," *NeurIPS*, vol. 31, 2018.

[7] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi, and Pascal Frossard, "Universal adversarial perturbations," in *CVPR*, July 2017.

[8] Chaoning Zhang, Philipp Benz, Tooba Imtiaz, and In So Kweon, "Understanding adversarial examples from the mutual influence of images and perturbations," in *CVPR*, 2020, pp. 14521–14530.

[9] Alex Krizhevsky, Geoffrey Hinton, et al., "Learning multiple layers of features from tiny images," 2009.

[10] Jaehoon Lee, Lechao Xiao, Samuel Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington, "Wide neural networks of any depth evolve as linear models under gradient descent," *NeurIPS*, vol. 32, 2019.

[11] Zhengdao Chen, "Multi-layer neural networks as trainable ladders of Hilbert spaces," in *ICML*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, Eds. 23–29 Jul 2023, vol. 202 of *Proceedings of Machine Learning Research*, pp. 4294–4329, PMLR.

[12] Lechao Xiao, Jeffrey Pennington, and Samuel Schoenholz, "Disentangling trainability and generalization in deep neural networks," in *ICML*. PMLR, 2020, pp. 10462–10472.

[13] Atish Agarwala and Yann Dauphin, "Sam operates far from home: eigenvalue regularization as a dynamical phenomenon," in *International Conference on Machine Learning*. PMLR, 2023, pp. 152–168.

[14] Shiye Lei, Fengxiang He, Yancheng Yuan, and Dacheng Tao, "Understanding deep learning via decision boundary," *IEEE TNNLS*, 2023.

[15] Huan Xiong, Lei Huang, Mengyang Yu, Li Liu, Fan Zhu, and Ling Shao, "On the number of linear regions of convolutional neural networks," in *ICML*. PMLR, 2020, pp. 10514–10523.

[16] Maithra Raghu, Ben Poole, Jon Kleinberg, Surya Ganguli, and Jascha Sohl-Dickstein, "On the expressive power of deep neural networks," in *ICML*. PMLR, 2017, pp. 2847–2854.

[17] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy, "Explaining and harnessing adversarial examples," in *ICLR 2015,*, Yoshua Bengio and Yann LeCun, Eds., 2015.

[18] Hanxiao Liu, Karen Simonyan, and Yiming Yang, "DARTS: differentiable architecture search," *CoRR*, vol. abs/1806.09055, 2018.

[19] Yuhui Xu, Lingxi Xie, Xiaopeng Zhang, Xin Chen, Guo-Jun Qi, Qi Tian, and Hongkai Xiong, "PC-DARTS: partial channel connections for memory-efficient architecture search," in *ICLR 2020*, 2020.

[20] Jisoo Mok, Byunggook Na, Hyeokjun Choe, and Sungroh Yoon, "Advrush: Searching for adversarially robust neural architectures," in *ICCV*, 2021, pp. 12322–12332.

[21] Yaguan Qian, Shenghui Huang, Bin Wang, Xiang Ling, Xiaohui Guan, Zhaoquan Gu, Shaoning Zeng, Wujie Zhou, and Haijiang Wang, "Robust network architecture search via feature distortion restraining," in *ECCV*, 2022, vol. 13665, pp. 122–138.

[22] Namhoon Lee, Thalaiyasingam Ajanthan, and Philip H. S. Torr, "Snip: single-shot network pruning based on connection sensitivity," in *ICLR 2019*, 2019.

[23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep residual learning for image recognition," in *IEEE CVPR*, 2016, pp. 770–778.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on March 25,2026 at 05:33:42 UTC from IEEE Xplore. Restrictions apply.